

Querying graphs with data modulo theories

Anonymous Author(s)

Abstract

Path-query languages over graph databases that combine data and topology reasoning have been the subject of many papers in the last decade. Existing proposals for a “data graph” model assume an abstract infinite set, wherein two elements can be checked for equality. Since paths in such a graph are simply *data words*, Libkin et al. considered various automata models over data words as path constraints, giving rise to extensions of Regular Path Queries (RPQs) and Conjunctive Regular Path Queries (CRPQs). We propose an extended data model allowing typed data (e.g. strings, integers, rationals) to be stored in the graph. The new data model addresses, among others, the lack of arithmetical reasoning support in the current model, e.g., “is someone that is located within 5km radius, younger/older by at most 2 years, and of friendship distance at most 4 available for a date?”. Main question we address: How to extend regular path languages to allow reasoning on infinite typed data, while preserving decidability of emptiness? We propose a first solution based on parametric automata modulo theories, which allow data reasoning modulo decidable logical theories (e.g. linear arithmetic, automatic relations) through the use of *parameters* as global value placeholders. The query evaluation problems for these extended (C)RPQ are shown to be decidable. Moreover, we show the combined and data complexity of (C)RPQs over various important logical theories and delineate cases where NL data complexity is achievable (e.g. integer/real linear arithmetic, automatic relations).

CCS Concepts: • Computer systems organization → Embedded systems; Redundancy; Robotics; • Networks → Network reliability.

Keywords: Data graphs, automata, complexity

ACM Reference Format:

Anonymous Author(s). 2018. Querying graphs with data modulo theories. In *Woodstock '18: ACM Symposium on Neural Gaze Detection, June 03–05, 2018, Woodstock, NY*. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/1122445.1122456>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Woodstock '18, June 03–05, 2018, Woodstock, NY

© 2018 Association for Computing Machinery.

ACM ISBN 978-1-4503-XXXX-X/18/06...\$15.00

<https://doi.org/10.1145/1122445.1122456>

1 Introduction

The past decade or so has witnessed a lot of theoretical and practical work in querying graph databases (see [2, 3, 45] for surveys), owing to their growing number of applications, e.g., in social networks, semantic web, and supply chain management. Indeed, graph query languages — such as Cypher [43] and Gremlin [36] — have been adopted by multiple vendors that provide graph database systems as their primary query languages, which led to a recent and ongoing standardization effort for a Graph Query Language (GQL) [1].

On the theoretical side, one important research avenue has been the development of “well-behaved” graph query languages (e.g. with good data complexity), which can reason about *topological* aspects of the graph. They are also known as *path query languages*, because topological information in the database typically amounts to querying the existence of paths satisfying certain constraints. For a social network database, one can take the query “are two given users of distance at most 5 in the graph?” as the simplest example of path queries. In 1995 Mendelzon and Wood [42] introduced the Regular Path Query (RPQ) language, which has been the foundation of many other query languages including Conjunctive Regular Path Queries (CRPQs) [16, 21, 24] or Extended CRPQ (ECRPQs) [6], to name a few. One of the main attractive theoretical features of these query languages is that they all possess NL (Nondeterministic Logspace) *data complexity* — i.e., for any fixed query — potentially allowing traversals of large graphs in practice (e.g. see [4]).

Combining data and topology: state of the art. The aforementioned path-query languages are unfortunately not suitable for *data-querying* in graph databases, which is bread and butter for relational database query languages. More precisely, in a typical graph database each node is usually associated with some data like names and ages of the users. In relational algebra (assuming a relational encoding of graph databases), one could easily express the query that outputs two node ids for users with the same ages. In the past decade, the importance of *combining data and topology* reasoning in graph query languages has been the subject of multiple works, e.g., [5, 29, 38, 39]. The importance of combining these two types of reasoning can be seen from the following query: find two people in the database with the same age and friend distance at most 5. In particular, walk logic (WL) [29] and regular expressions with memory (REM) [39] emerged as two initial solutions for overcoming this limitation. While RPQ using REM and register automata as path constraint has NL-complete data complexity [38, 39], WL was shown to have non-elementary data complexity [5]. The crucial

observation in [38, 39] is that paths in a graph database can be construed as a *data word*, which enables one to apply automata over data words (e.g. register automata [30], REM [39], data automata [14], etc.) as path constraints in a path-query. Interestingly, many different automata models over data words (e.g. data automata) increase the data complexity of RPQ from NL to NP-hard [38].

The need for reasoning about concrete data. One of the main limitations of the aforementioned solutions for graph databases that combine data and topology is that data elements in the graphs can only be checked for (in)equality. This is very restrictive in practice. Practical database query languages typically allow *concrete data* (e.g. numbers, strings) and allow richer data comparisons (e.g. inequality of numbers). Consider, for instance, the query “does there exist someone that is available for a date located within a 5km radius, younger/older by at most 2 years, that is of friendship distance at most 5?” Alternatively, take the example of “does there exist two connected users whose names are of edit distance at most 3?”, which is motivated by the problem of “unclean” real-world data and the data cleaning problem (see [23] for a survey). Both of these lie outside the scope of existing graph query languages that combine data and topology, since they require arithmetic and string reasoning.

Contributions. This paper aims to investigate the possibility of extending the current graph data model with reasoning about concrete data. To this end, we first extend the existing data model by allowing *typed data* (e.g. strings, integers, rationals) to be stored in the graph, as in Figure 1. This takes us immediately to the first main open question: *how to extend regular languages to allow reasoning of typed infinite data, while preserving decidability of emptiness?* While the observation by Libkin et al. [38, 39] that paths can be treated as data words generalize to typed data, there is a lack of sufficiently general automata model over data words allowing general data types but with a decidable language emptiness. In particular, many existing decidable automata models and logics over data words become undecidable when extended even with a linear order in the data [8, 14, 19, 20], not to mention linear arithmetics. We present here a first solution to this problem: *parametric automata modulo theories (PAMT)*, which is also called $\mathbb{L}$ -pNFA for any decidable theory $\mathbb{L}$.

PAMT are inspired by automata modulo theories (AMT) and similar formulations called symbolic automata [17, 40, 41], which were introduced to exploit the power of Satisfiability Modulo Theories (SMT) and fast SMT-solvers to the world of automata. As an illustration, suppose we want to recognize the set of sequences containing only even numbers. Then, one can describe this by the regex-like expression $(x \equiv_2 0)^*$ or its equivalent automaton modulo the theory $\mathbb{Z}_{LA}$ of integer linear arithmetic (a.k.a. Presburger Arithmetic). That is, instead of allowing only concrete letters in the alphabet, we use the SMT formula $x \equiv_2 0$ to capture the infinitely

many automata transitions $q \rightarrow_i q$, for every even number i . AMT are known to satisfy virtually all decidability and closure properties that are well-known for standard finite automata over finite alphabets [17, 41]. Unfortunately, *AMT are a very weak model for querying data graphs*, which cannot even recognize the set of sequences i , i of length two containing the same number $i \in \mathbb{Z}$, i.e., they cannot meaningfully relate one node to another node in the query.

PAMT overcome the aforementioned problem by means of *parameters*, which can be thought of as global value placeholders, or as (non-deterministically instantiated) read-only registers. To recognize a sequence of integers $m_1, \dots, m_n$ such that the difference between minimum and the maximum is less than 2, one can use two parameters $p_{\min}, p_{\max}$ and assert that $p_{\min} < p_{\max} \wedge p_{\max} - p_{\min} \leq 2$ and that each number in the sequence is in between $p_{\max}$ and $p_{\min}$; more precisely, the formula in the only transition is $\varphi(p_{\min}, p_{\max}, x) := p_{\min} \leq x \wedge x \leq p_{\max}$. Notice that this is a formula in $\mathbb{Z}_{LA}$. This idea can be easily extended with a finite alphabet component (as in the setting of data graph in [38, 39]), as well as other decidable theory (e.g. over integers, rationals, strings, or a combination thereof). Using other appropriate decidable theories over strings (e.g. automatic relations [7, 10, 12]), one can also similarly express the aforementioned edit distance example.

Our contributions can be summarized as follows:

1. We introduce $\mathbb{L}$ -pNFA as suitable language recognizers for data words modulo decidable theories $\mathbb{L}$, and show decidability of emptiness.
2. The query evaluation problem of RPQ with $\mathbb{L}$ -pNFA is shown to have pretty much the same combined complexity as that of satisfiability over $\mathbb{L}$. For the theory $\mathbb{L}$ of integer/rational linear arithmetic ($\mathbb{Z}_{LA}, \mathbb{Q}_{LA}$), the combined complexity is NP-complete, whereas it is PSPACE-complete for existential theory of automatic relations.
3. For data complexity, we show the important roles of *data size* (wrt. size $\|G\|$ of the underlying graph G) and *theory choice*:
 - Over $\mathbb{Z}_{LA}$ and $\mathbb{Q}_{LA}$, data complexity is NL.
 - Over existential theory over strings using automatic relations, the data complexity is NP-hard. This can be lowered to NL-complete by restricting the formulas used or by assuming the *log-size hypothesis (LogH)*: size of data is $O(\log \|G\|)$. This assumption is satisfied in most practical graph databases, where the number of nodes in G dwarfs the size of data in a single node.
 - Over existential theory of integer linear arithmetic with divisibility (coNEXP and is NP-hard [37, 44]), data complexity is NP-hard, even under LogH.
 - Over existential theory over strings with substring-of relation [35], data complexity is NP-hard under LogH.
4. We pinpoint some extensions: (1) CRPQ with $\mathbb{L}$ -pNFA as path constraints, (2) combining theories to preserve NL data complexity, and (3) aggregations.

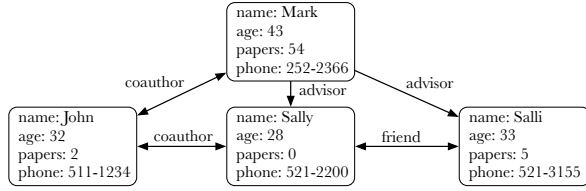


Figure 1. A typed data graph.

Organization. We introduce the data model (i.e. typed data graphs) and query language (i.e. RPQ with $\mathbb{L}$ -pNFA as path constraints) in Section 2. We prove decidability of $\mathbb{L}$ -pNFA and combined complexity results in Section 3. We prove data complexity results in Section 4. After showing possible extensions in Section 5, we conclude in Section 6.

2 Data model and queries

2.1 Typed data graphs

We define the notion of typed data graphs and use them as our data models. Roughly speaking, typed data graphs are simply data graphs [38, 39], whose (infinite) domain contain tuples of elements, each is associated with a type and a size. For example, such an element could be the number 32 that is of the integer type (INT) and has size 5 (i.e. the number of bits needed to represent 32). *For the sake of simplicity and clarity of the exposition, we assume that each such tuple satisfies the same “schema”, e.g., the tuple is indexed by the attributes x, y, z , where x and y are of the string type (e.g. for first and last names of a person), and z is of the integer type (e.g. representing the age of a person).*

We now formalize the above intuitive notion. Fix a finite set $\mathcal{T} = \{t_1, \dots, t_k\}$ of “types”. We assume that each type t_i is associated with a domain $\mathcal{D}_i$ of elements. For example, one could have a type INT that is associated with the domain $\mathbb{Z}$. We assume a countable set $Atts = \{x, y, z, \dots\}$ of attributes, where each attribute is assigned a specific type. A *schema* is defined to be a finite set $S \subseteq Atts$ of attributes. For the purpose of measuring complexity, we assume a size function $|\cdot| : \mathcal{D} \rightarrow \mathbb{N}$, where $\mathcal{D} := \bigcup_{i=1}^k \mathcal{D}_i$.

We use $\mathcal{V}(S)$ to denote the set of valuations $v : S \rightarrow \mathcal{D}$ such that the type of $v(x)$ agrees with the type of x .

Definition 2.1. A *typed data graph* (with schema S over an alphabet Σ) is a triplet $G = (V, E, \lambda)$, where V is a finite set of nodes, $E \subseteq V \times \Sigma \times V$ a finite set of Σ -labeled edges on V , and $\lambda : V \rightarrow \mathcal{V}(S)$ is a function associating each node with tuples of data over S .

This definition extends the notion of *data graphs* [38], where no types are needed. A crucial fundamental notion for typed data graphs G is that of (G) -paths.

Definition 2.2. A G -path from s to t , where $s, t \in V$, is simply an alternating sequence $v_0 a_1 \dots a_n v_n$ of vertices and

edge labels of G (i.e. each $v_i \in V$ and each $a_j \in \Sigma$) such that $(v_i, a_{i+1}, v_{i+1}) \in E$ for each $i \in [0, n-1]$.

2.2 Data Path Queries

The notion of data path queries over typed data graphs extends easily from the non-typed data graph setting [38]. We fix a recognizer model M for (typed) data languages L as a path constraint. Let $\Sigma_{\#} = \Sigma \cup \{\#\}$, where $\# \notin \Sigma$ is a fixed “dummy symbol”. A *data language* (over schema S) over the finite alphabet $\Sigma_{\#}$, is a subset of $(\Sigma_{\#} \times \mathcal{V}(S))^*$, i.e., a set of sequences of elements of $\Sigma_{\#} \times \mathcal{V}(S)$. The *regular path query* (RPQ) associated to L is the query of the form $x \rightarrow_L y$. The fundamental decision problem we study here is the *query evaluation problem (QEP)* for M -RPQ: given a typed data graph $G = (V, E, \lambda)$ (over schema S), two nodes $s, t \in V$, and a data language L (over schema S) represented by an M -recognizer, decide if there is a G -path $\pi = v_0 a_1 \dots a_n v_n$ from s to t (i.e. $s = v_0$ and $t = v_n$) such that

$$(\#, \lambda(v_0))(a_1, \lambda(v_1)) \dots (a_n, \lambda(v_n)) \in L.$$

As in [38], we will vary the recognizer model M for data languages when discussing the query evaluation of M -RPQ. In the next subsection, we will define various recognizer models for typed data languages by means of various Satisfiability Modulo Theories (SMT), and in Section 2.4 determine the complexity of query evaluation for various theories. In the special case where the recognizer ignores the data values and imposes a regular constraint on the Σ -labels, we obtain the classical definition of RPQ [42].

2.3 Theories of strings and numbers

Here we assume a given multi-sorted first-order structure $\mathfrak{S}$ over some vocabulary σ , whose domains are $\mathcal{D}_1, \dots, \mathcal{D}_k$. We consider some *decidable* fragment $\mathbb{L}$ of first-order logic (FO) – e.g. the quantifier-free fragment – over $\mathfrak{S}$, in the following sense: it is decidable to check if, given a formula $\varphi(\bar{x})$, there exists a valuation v such that $\mathfrak{S}, v \models \varphi$. We assume that the function χ maps each first-order variable x to its type (i.e. one of $t_1, \dots, t_k$). Among the set of variables, we single out the set $\mathcal{P} := \{p_1, p_2, \dots\}$ and call them *parameters*, which has infinitely many variables with each of the types $t_1, \dots, t_k$. Any variable that is not in $\mathcal{P}$ is called *normal* or *non-parametric*. Given a set X of variables, let $\mathbb{L}[X]$ be the set of all formulas whose free variables are included in X . Whenever we write $\varphi(\bar{x}; \bar{p})$ we refer to a formula having $\bar{x}\bar{p}$ as free variables, of which $\bar{p}$ are parameters.

Linear arithmetic. We define the theories of integer and rational linear arithmetic $\mathbb{Z}_{LA}$, $\mathbb{Q}_{LA}$. The domain of $\mathbb{Z}_{LA}$ (resp. $\mathbb{Q}_{LA}$) is the set of all integer numbers $\mathbb{Z}$ (resp. rational numbers $\mathbb{Q}$). It is defined as the FO over $\mathbb{Z}$ (resp. $\mathbb{Q}$), in the vocabulary of: addition function $(\cdot + \cdot)$, inequality relation $(\cdot \leq \cdot)$, constants $c \in \mathbb{Z}$ (resp. $c \in \mathbb{Q}$), and scalar multiplication $\cdot_c$ for each $c \in \mathbb{Z}$ (resp. for each $c \in \mathbb{Q}$). It is possible to allow all real numbers in the domain of $\mathbb{Q}_{LA}$ but still keep

rational constants, but this will result in exactly the same theory (i.e. not first-order distinguishable). For the purpose of quantifier elimination it is also necessary to add to $\mathbb{Z}_{LA}$ the k -congruence (binary) relation $\equiv_k$ for each $k \in \mathbb{Z}$ with $k \geq 1$, where $x \equiv_k y$ iff $x = y + kc$ for some $c \in \mathbb{Z}$. For example, $\exists x(x + 2z > y \wedge y \equiv_7 0 \wedge z \equiv_2 0)$ is an existential formula over $\mathbb{Z}_{LA}$ with two free variables y, z . The interpretations of these relations and functions are standard, and so are the definition of valuations and formula satisfiability and equivalence (e.g. see [22, 33]). Note that $\mathbb{Z}_{LA}$ is often referred to as Presburger Arithmetic. It is known (e.g. see [22, 33]) that $\mathbb{Z}_{LA}$ and $\mathbb{Q}_{LA}$ admit quantifier elimination: every formula can be algorithmically converted to an equivalent quantifier-free formula over the theory. In particular, all these theories are decidable. It is common (e.g. in SMT, see [33]) to restrict to quantifier-free and existential formulas primarily because of the prohibitive complexity of the full FO theory of $\mathbb{Z}_{LA}$ and $\mathbb{Q}_{LA}$ (e.g. see [32]). The quantifier-free and the existential fragments of these theories are well-known to be NP-complete (e.g. [27, 33]).

Adding divisibility. One possible extension of $\mathbb{Z}_{LA}$ is to add the divisibility operator $(\cdot \mid \cdot)$. It is known that the existential fragment of $\mathbb{Z}_{LA}$ extended with divisibility, denoted by $\exists \mathbb{Z}_{LA}^{div}$, is still decidable in NEXP (nondeterministic exponential time) but is NP-hard [37, 44].

Automatic relations. Let Σ be a finite alphabet. The theory of automatic relations $\mathbb{L}_{Aut}$ is defined as first-order logic over the domain of all finite words Σ^* with the prefix relation $(\cdot \leq \cdot)$, equal-length relation ($eq-len(\cdot, \cdot)$) and last-letter relations ($lst_a(\cdot)$, for every $a \in \Sigma$). For example,

$$\begin{aligned} \varphi(x_1, x_2) = & \exists x_3 \text{ } lst_a(x_3) \wedge (x_3 \leq x_1) \wedge (x_3 \leq x_2) \wedge \\ & \forall z(x_3 < z \rightarrow \neg(z \leq x_1 \wedge z \leq x_2)) \end{aligned}$$

expresses the relation consisting of all $(w_1, w_2) \in \Sigma^* \times \Sigma^*$ such that the largest common prefix of w_1 and w_2 ends with a letter a .

Automatic relations can also be defined by means of automata. Let $\perp \notin \Sigma$ be a special 'blank' symbol, and let $\Sigma_\perp$ be $\Sigma \cup \{\perp\}$. Given words $w_1, \dots, w_\ell$ over Σ , their *convolution* $w_1 \otimes \dots \otimes w_\ell$ is the smallest word over $(\Sigma_\perp)^\ell$ such that for every $i \leq \ell$, its projection onto the i -th component yields a word from $w_i \cdot \{\perp\}^*$. For example, $aab \otimes c \otimes bb = (a, c, b)(a, \perp, b)(b, \perp, \perp)$. A k -ary word relation $R \subseteq (\Sigma^*)^k$ is *automatic* if $\{w_1 \otimes \dots \otimes w_k : (w_1, \dots, w_k) \in R\}$ is a regular language over $\Sigma_\perp^\ell$. In light of this definition, we assume that k -ary synchronous relations are represented as NFA (and hence whose satisfiability is in NL). If one considers positive existential formulas of atoms which are automatic relations represented by NFA, the satisfiability rises from NL to PSPACE, since it becomes, essentially, equivalent to the PSPACE-complete problem of emptiness of intersection of regular languages [31].

It is known that $\mathbb{Z}_{LA}$ is an automatic structure and $\mathbb{Q}_{LA}$ is an ω -automatic structure [11]. In particular, for every formula $\varphi(x_1, \dots, x_k) \in \mathbb{Z}_{LA}$, there exists an (effective) automatic k -ary relation over the alphabet $\Sigma = \{0, 1, +, -\}$ consisting of all tuples $(w_1, \dots, w_k)$ such that $\mathbb{Z}, [w_1], \dots, [w_k] \models \varphi$, where $[w_i] \in \mathbb{Z}$ is the number represented by w_i , using a standard (binary) representation (e.g., $[-01001] = -9$, assuming a length 4 encoding) [15]. Similarly, for each formula $\varphi(x_1, \dots, x_k) \in \mathbb{Q}_{LA}$, there exists an ω -automatic k -ary relation over the alphabet $\Sigma = \{0, 1, +, -, \star\}$, where $\star$ is the decimal separator. An ω -automatic relation is a relation on ω -words that can be represented by an NFA over Σ^k with a Büchi accepting condition [13]. Numbers are also encoded in a standard way, for example, $5/2$ has two possible encodings with integer-part of length 3: $+010\star 1010^\omega$ and $+010\star 1001^\omega$.

Example 2.3. One important automatic relation is the binary relation $d_c(x, y)$ that asserts that the edit distance between strings x and y is at most c . Intuitively, the edit distance of two strings is the least number of insertions, deletions, and letter substitution that have to be applied to obtain y from x . It is known (e.g. see [6, 25]) that d_c for every fixed $c \in \mathbb{N}$ is an automatic relation. This notion was already used to motivate ECRPQ [6].

Substring theory. A widely-used string relation is the substring (a.k.a. factor, infix) relation, that is, the relation $v \leq_{substr} u$ holding if $u = w_1 \cdot v \cdot w_2$ for some strings w_1, w_2 . This is, notably, not an automatic relation. While the full first-order theory over $\leq_{substr}$ is in general undecidable, its quantifier-free fragment is decidable (and even its existential fragment) [35]. We will then consider $\mathbb{L}_{substr}$ to be the quantifier-free theory over $\leq_{substr}$.

Representation and data structure of values. We assume that numbers (integers and rationals) are represented in the database as described above (i.e., as a finite sequence of bits of the form $\pm w$ in the case of integers, or as an expression with ω of the form $\pm u \star v(w)^\omega$ in the case of rationals), and they can be read as a chained list of digits from left to right. The *size* of each value is the number of symbols representing these expressions.

2.4 Path Languages

We finally define our general setting of regular path languages under theories. Fix a finite alphabet Σ , a schema S , and a finite set $\bar{p}$ of parameters. Abusing notation, we identify attributes in S with non-parametric variables $\bar{x}$ that refer to them. A *parametric nondeterministic finite automaton modulo $\mathbb{L}$* (written $\mathbb{L}$ -pNFA for short) is simply an NFA, but each transition is labeled instead by $(a, \varphi(\bar{x}; \bar{p}))$, where $a \in \Sigma_\#$ and $\varphi(\bar{x}; \bar{p})$ a formula in $\mathbb{L}$. More precisely, an $\mathbb{L}$ -pNFA with parameters $\bar{p}$ is a tuple $\mathcal{A} = (R, Q, q_0, F, \Delta)$, where Q is a finite set of states, $q_0 \in Q$ an initial state, $F \subseteq Q$ a set of final states, R is a formula $\varphi(\cdot; \bar{p}) \in \mathbb{L}[\mathcal{P}]$, and

$\Delta \subseteq Q \times (\Sigma_{\#} \times \mathbb{L}) \times Q$, where the free parametric (resp. non-parametric) variables of each ψ in some $(q, (a, \psi), q') \in \Delta$ must be among $\bar{p}$ (resp. $\bar{x}$). For simplicity, and without loss of generality, we henceforth assume that in fact $\psi(\bar{x}; \bar{p})$ has exactly $\bar{x}$ as non-parametric free variables (remember we identify each variable of $\bar{x}$ with an attribute of S) and $\bar{p}$ as parameters. The language $\mathcal{L}(\mathcal{A})$ recognized by $\mathcal{A}$ is defined as follows. Take a path $\pi : q_0 \rightarrow_{(a_1, \psi_1(\bar{x}_1; \bar{p}))} \dots \rightarrow_{(a_n, \psi_n(\bar{x}_n; \bar{p}))} q_n$ in Q with $q_n \in F$. Consider relabeling the non-parametric variables $\bar{x}$ in $\psi_1, \dots, \psi_n$ by new pairwise disjoint variables $\bar{x}_1, \dots, \bar{x}_n$ (preserving the types). If there is a valuation v such that

$$\mathfrak{S}, v \models \varphi(\bar{p}) \wedge \bigwedge_{i=1}^n \psi_i(\bar{x}_i; \bar{p}),$$

then we add the following data word to $\mathcal{L}(\mathcal{A})$:

$$(a_1, v(\bar{x}_1)) \dots (a_n, v(\bar{x}_n)).$$

We can now consider the RPQ-based query language based on $\mathbb{L}$ -pNFA. We define $\mathbb{L}$ -RPQ to be M -RPQ, where $M = \mathbb{L}$ -pNFA, that is, RPQ with $\mathbb{L}$ -pNFA as a path constraint.

2.5 Examples

Based on the example in Figure 1, here are some examples showing the different sorts of queries that are expressible.

1. “find two nodes with similar age (± 3) linked by a friendship path”
2. “find two nodes with similar names (edit distance ≤ 4) linked by a friendship path”
3. “find two individuals linked by a friendship path of people sharing the same area code (in phone number)”
4. “find two nodes with similar names (edit distance ≤ 4) and similar age (± 3) linked by the coauthor relationship of nodes the same number of papers”.
5. “find every two individuals linked by a coauthor path of people where each person is of age ≥ 20 and the average age is less than 40”.

We can express Example 1 as $\mathbb{Z}_{LA}$ -pNFA $\mathcal{A} = (R, Q, q_0, F, \Delta)$ with three states $Q = \{q_0, q_1, q_F\}$, one parameter p , $F = \{q_F\}$, and $R = \text{true}$. The transitions are: $(q_0, (\#, p = \text{age}), q_1)$, $(q_1, (a, \text{true}), q_1)$, $(q_1, (a, |p - \text{age}| \leq 3, q_F))$. Here a refers to friend label. The other examples can be done in a similar way but varying the constraints in the logical theory. Examples 2 and 3 can be done over $\mathbb{L}_{Aut}$ -RPQ. Example 4 can be done over $\mathbb{L}$ -RPQ using a mixed theory, whereas Example 5 can be done using aggregation (see Section 5 for these).

3 Combined complexity

We show that the evaluation of $\mathbb{L}$ -RPQ is decidable whenever $\mathbb{L}$ is decidable, and further that one can derive complexity bounds from the complexity of $\mathbb{L}$. Let us first start with the emptiness problem for pNFA modulo theories.

Theorem 3.1. *For every decidable $\mathbb{L}$, the emptiness problem for $\mathbb{L}$ -pNFA is decidable.*

Proof. For $\mathcal{A} = (R, Q, q_0, F, \Delta)$, we claim that $\mathcal{L}(\mathcal{A}) \neq \emptyset$ iff there exists a simple path $q_0 \rightarrow_{(a_1, \psi_1(\bar{x}_1; \bar{p}))} \dots \rightarrow_{(a_n, \psi_n(\bar{x}_n; \bar{p}))} q_n$ in Δ with $q_n \in F$ such that

$$\mathfrak{S} \models \exists \bar{p} \bar{x}_1 \dots \bar{x}_n R(\bar{p}) \wedge \bigwedge_{i=1}^n \psi_i(\bar{x}_i; \bar{p}), \quad (1)$$

where the $\bar{x}_i$ the pairwise disjoint renamings of the variables $\bar{x}$. This would immediately give us an algorithm for checking $\mathcal{L}(\mathcal{A}) \neq \emptyset$: guess a simple path from q_0 to some final state, and the rest is obtained from decidability of $\mathbb{L}$.

From right to left is obvious. So, we prove the converse. Assume that $\mathcal{L}(\mathcal{A}) \neq \emptyset$ and so there exists a (not necessarily simple) path $\pi : q_0 \rightarrow_{(a_1, \psi_1(\bar{x}_1; \bar{p}))} \dots \rightarrow_{(a_n, \psi_n(\bar{x}_n; \bar{p}))} q_n$ with $q_n \in F$ such that (1) holds.

A standard fact in graph theory states that π contains a subpath $\pi' : q_0 \rightarrow^* q_n$ that is simple. We collect the labels $\psi_i(\bar{x}_i; \bar{p})$ that appears in π' , say, $\psi_{i_1}(\bar{x}_{i_1}; \bar{p}), \dots, \psi_{i_m}(\bar{x}_{i_m}; \bar{p})$. We then have

$$\begin{aligned} \mathfrak{S} \models \exists \bar{p} \bar{x}_1 \dots \bar{x}_n R(\bar{p}) \wedge \bigwedge_{i=1}^n \psi_i(\bar{x}_i; \bar{p}) \\ \models \exists \bar{p} \bar{x}_{i_1} \dots \bar{x}_{i_m} R(\bar{p}) \wedge \bigwedge_{j=1}^m \psi_{i_j}(\bar{x}_{i_j}; \bar{p}). \quad \square \end{aligned}$$

One corollary of Theorem 3.1 is a generic theorem for combined complexity of query evaluation for data graphs over $\mathbb{L}$ -pNFA models.

Theorem 3.2. *Suppose that the satisfiability problem for $\mathbb{L}$ is $C \in \{\text{NP}, \text{PSPACE}\}$ or $C \supseteq \text{EXP}$. Then, the combined complexity of $\mathbb{L}$ -RPQ evaluation is in C .*

Proof. Let $G = (V, E, \lambda)$ be a typed graph with start and end nodes $s, t \in V$. Let the given query be $x \rightarrow_L y$, where L is an $\mathbb{L}$ -pNFA $\mathcal{A} = (R, Q, q_0, F, \Delta)$. To check $s \rightarrow_L t$, we take a product construction of G with $\mathcal{A}$. That is, we construct a new $\mathbb{L}$ -pNFA $\mathcal{A}'$ but with no non-parametric variables (because they will be instantiated by the corresponding values in G). That is, $\mathcal{A}' = (R, Q', q_0, F', \Delta')$, where

- $Q' = (V \times Q) \cup \{q_0\}$
- $F' = \{t\} \times F$
- Δ' is the union of
 - all $(q_0, (\#, \psi[\bar{x} \mapsto \lambda(s)](\bar{p})), (s, q))$ such that there is some $(q_0, (\#, \psi(\bar{x}; \bar{p})), q)$ in Δ , where $\psi[\bar{x} \mapsto \lambda(s)]$ stands for the instantiation of $\bar{x}$ according to $\lambda(s)$; and
 - all $((v, q), (a, \psi[\bar{x} \mapsto \lambda(v')](\bar{p})), (v', q'))$ such that $(v, a, v') \in E$ and $(q, (a, \psi(\bar{x}; \bar{p})), q') \in \Delta$.

Notice that this is a polynomial-time reduction (in fact, a logspace reduction). The combined complexity in C is obtained by simply invoking the algorithm in the proof of Theorem 3.1. In particular, when C is NP (resp. PSPACE), we obtain immediately an NP (resp. NPSpace, which is just PSPACE) algorithm. When C subsumes EXP, the simple

accepting path in $\mathcal{A}'$ can be found by brute force of deterministic enumeration, giving us a complexity of C for checking emptiness for $\mathcal{A}'$. $\square$

Corollary 3.3. *The combined complexity of $\mathbb{Z}_{LA}$ -RPQ and $\mathbb{Q}_{LA}$ -RPQ is NP-complete, assuming formulas are represented as existential formulas.*

Corollary 3.4. *The combined complexity of $\exists\mathbb{Z}_{LA}^{div}$ -RPQ is in NEXP and NP-hard.*

Corollary 3.5. *Assuming formulas are conjunctions of automatic relations represented as NFA, the combined complexity of $\mathbb{L}_{Aut}$ -RPQ is PSPACE-complete,*

4 Data Complexity

We now focus on the main theme of the paper, namely the data complexity of $\mathbb{L}$ -RPQ for the theories we have introduced. The bad news is that for all the theories $\mathbb{L}$ we have considered in the previous sections, the data complexity of $\mathbb{L}$ -RPQ evaluation is NP-hard, with one exception: linear arithmetic.

#-loop assumption Remember that the special symbol $\#$ in the definition of RPQ is there because there is a mismatch between the number of edges and the number of vertices in a path, and we wish to express properties of both the edges and the vertices. To simplify the technical developments in this section, we will however assume that all the graph databases we deal with have a $\#$ -labelled self-loop at each vertex. This is by no means with any loss of generality, and it is just a simple trick to avoid dealing with the first element of the path as a special case in each of the proofs.

4.1 Numeric domain

Theorem 4.1. *Data complexity of $\mathbb{Z}_{LA}$ -RPQ and $\mathbb{Q}_{LA}$ -RPQ evaluation is NL-complete.*

Proof. Fix a data graph $G = (V, E, \lambda)$, and a $\mathbb{Z}_{LA}$ -pNFA $\mathcal{A} = (R, Q, q_0, F, \Delta)$ with parameters $\bar{p} = p_1, \dots, p_k$. The $\mathbb{Z}_{LA}$ -RPQ asks for the existence of a path from $s \in V$ to $t \in V$ in G in the language of $\mathcal{A}$. Let us assume that the tuple of non-parametric variables (given by the schema) is $\bar{x} = x_1, \dots, x_\ell$.

By quantifier elimination, it can be assumed that for some $d \in \mathbb{N}$ every formula $\varphi(\bar{x}; \bar{p})$ of $\mathcal{A}$ in this theory is expressed as a disjunction of quantifier-free formulas of the form

$$\bigwedge_{i=1}^n h_i(\bar{p}) \sim_i h'_j(\bar{x})$$

where $\sim_i \in \{<, >, =, \equiv_d\}$, each of $h_i(\bar{p})$ is a linear combination of the form $\sum_{p \in \bar{p}} a_p p$, and $h'_j(\bar{x})$ is a linear combination of the form $\sum_{x \in \bar{x}} a_x x + c$, where $c \in \mathbb{Z}$ (resp. $c \in \mathbb{Q}$). Observe that d is the same for all formulas of $\mathcal{A}$.

Our NL algorithm is a nondeterministic logspace machine on the “product” of G and $\mathcal{A}$, starting at the state (s, q_0) . For each possible (in)equality and modulo arithmetic $\psi :=$

$h(\bar{p}) \sim h'(\bar{x})$, we keep a pointer n_ψ (of logspace) to a vertex of the database. All pointers are initialized to ‘?’, meaning “undefined”. Suppose that the current configuration is (v, q) . The algorithm nondeterministically guesses (v', q') such that $(v, a, v') \in E$ and $(q, (a, \theta(\bar{x}; \bar{p})), q') \in \Delta$. It further guesses a disjunct θ' in θ , and does the following, for each $\psi := h(\bar{p}) \sim h'(\bar{x})$ appearing in θ' :

1. If $n_\psi = ?$, set $n_\psi := v'$
 2. If $n_\psi \neq ?$ and $\sim$ is $=$, verify $h'(\lambda(n_\psi)) = h'(\lambda(v'))$. Otherwise, reject immediately.
 3. If $n_\psi \neq ?$ and $\sim$ is $\equiv_d$, verify $h'(\lambda(n_\psi)) \equiv_d h'(\lambda(v'))$. Otherwise, reject immediately.
 4. If $n_\psi \neq ?$, $\sim$ is $<$, and $h'(\lambda(v')) < h'(\lambda(n_\psi))$, set $n_\psi := v'$.
 5. If $n_\psi \neq ?$, $\sim$ is $>$, and $h'(\lambda(v')) > h'(\lambda(n_\psi))$, set $n_\psi := v'$.
- Observe that the tests above of the form $h'(\lambda(n_\psi)) \sim h'(\lambda(v'))$ can be done in NL, by first computing the (ω) -automatic relation corresponding to the formula with 2ℓ free variables $h'(y_1, \dots, y_\ell) \sim h(z_1, \dots, z_\ell)$ (represented as DFA over $\Sigma_\perp^{2\ell}$), and then reading synchronously the tuples of values from n_ψ and v' testing whether they belong to the relation.

Finally, let F be the conjunction of $h(\bar{p}) \sim h'(\lambda(n_\psi))$ for every $\psi = h(\bar{p}) \sim h'(\bar{x})$ such that $n_\psi \neq ?$. If $v = t$ and $q' \in F$, then the algorithm stops, and accepts iff $F \wedge R$ is satisfiable. As before, we can check satisfiability for $F \wedge R$ by considering the (ω) -automatic relation corresponding to the conjunction of $h(\bar{p}) \sim h'(\bar{y}_\psi)$ for every $\psi = h(\bar{p}) \sim h'(\bar{x})$ such that $n_\psi \neq ?$, in such a way that variables $\bar{y}_\psi$ and $\bar{y}_{\psi'}$ are disjoint for every two distinct ψ, ψ' . We can thus read synchronously the values of all $\lambda(n_\psi)$ while guessing values for the $\bar{p}$ parameters to test that they are in the relation, and hence that $F \wedge R$ is satisfiable. Otherwise, the algorithm goes to (v', q') and repeats the above process. If at some point the algorithm stops, the satisfying assignment for the parameters $\bar{p}$ will be a witness for acceptance by $\mathcal{A}$ of the traversed underlying G -path from s to t . Conversely, an accepting run of $\mathcal{A}$ on an s -to- t path in G gives rise to a run of the nondeterministic logspace algorithm and a satisfying valuation of the parameters in $(F \wedge R)(\bar{p})$. $\square$

As soon as we add divisibility, evaluation becomes NP-hard.

Theorem 4.2. *Data complexity of $\exists\mathbb{Z}_{LA}^{div}$ -RPQ evaluation is NP-hard.*

4.2 Strings

For strings, let us start by showing that the substring theory is hard for NP.

Theorem 4.3. *Data complexity of $\mathbb{L}_{substr}$ -RPQ is NP-hard.*

Unfortunately also the theory of automatic relations is hard for NP.

Theorem 4.4. *Data complexity of $\mathbb{L}_{Aut}$ -RPQ evaluation is NP-hard.*

In view of this NP-hardness result, we show two ways in which the bound can be improved for $\mathbb{L}_{Aut}$:

1. by restricting the formulas used, or
2. by restricting the size of the graph data values.

1. Restricting the formulas. The result of Theorem 4.4 above begs the question of whether there are restrictions of $\mathbb{L}_{Aut}$ which make the RPQ evaluation in NL. We shown one such restriction. Let us call an automatic formula ψ to be *weak* if it is of the form

$$\psi(\bar{x}; \bar{p}) = \bigvee_{i \in I} \psi_i(\bar{x}; \bar{p}) \wedge \psi'_i(\bar{x};) \wedge \bigwedge_{j \in J} \xi_{i,j} \quad (2)$$

where ψ_i and ψ'_i are arbitrary formulas, and the $\xi_{i,j}$ formulas are either

- atoms of the form $p \leq x, p \geq x$ or
- $p \sim x$ for any automatic partial equivalence relation $\sim$ (in particular, $\sim$ may be *eq-len*),

where $p \in \bar{p}$ and $x \in \bar{x}$.

Indeed, the query used in the hardness proof of Theorem 4.4 is not weak (cf. Appendix). For example, the relation of all pairs (u, v) sharing the same $\{a, b\}$ -prefix is an automatic equivalence relation, or the relation defined by the formula $\varphi(x, y) = \exists z \psi(x, z) \wedge \psi(y, z)$ for any arbitrary ψ .

Let us define a $\mathbb{L}_{Aut}$ -pNFA to be weak, if every formula contained in the transition relation is weak. In particular, the guard formula R can be any arbitrary $\mathbb{L}_{Aut}$ -formula. Let weak $\mathbb{L}_{Aut}$ -RPQ be the class of RPQ based on weak $\mathbb{L}_{Aut}$ -pNFA. In particular, example 3 from §2.5 is in weak $\mathbb{L}_{Aut}$ -RPQ

Theorem 4.5. *Data complexity of weak $\mathbb{L}_{Aut}$ -RPQ is NL-complete.*

2. Logarithmic size data. Observe that the lower bound of Theorem 4.4 above uses a graph database with very big strings of size linear in the number of nodes. However, this linear dependence is arguably not reasonable for real-life graph databases. In numerous examples of graph databases, it is often the case that the sizes of values contained in the nodes are comparatively very small with respect to the size of the data graph. Motivated by this, we will now consider working with typed data graphs G such that the sizes of the data values in the nodes are at most logarithmic in the number of edges of G . We refer to this as the *log-size assumption*. For instance, if each node in G has an integer data value, then the sizes of these integers, namely the lengths of their binary encodings, are at most logarithmic in the number of edges of G . This does not exclude, for example, that each node can have a unique data value.

However, for the theory of Automatic relations $\mathbb{L}_{Aut}$, the log-size assumption allows an NL upper bound.

Theorem 4.6. *Data complexity of $\mathbb{L}_{Aut}$ -RPQ evaluation is NL-complete under the log-size assumption.*

Proof. Let N be the maximum length of a word in G ; recall it is logarithmic in the number of edges of G . Let $\mathcal{A}$ be a

$\mathbb{L}_{Aut}$ -pNFA and G a graph under log-size regime. Consider an accepting path from vertex s to vertex t in G , as witnessed by a G -path $\pi = v_0 a_1 \cdots a_n v_n$ from s to t and some transitions $(q_0, (a_0, \psi_0(\bar{x}; \bar{p})), q_1), \dots, (q_n, (a_n, \psi_n(\bar{x}; \bar{p})), q_{n+1})$ in $\mathcal{A}$ such that q_0 and q_{n+1} are initial and final respectively, and $a_0 = \#$. Observe that this means that the formula $\varphi(\bar{x}; \bar{p}) = R(\bar{x}; \bar{p}) \wedge \bigwedge_i \psi(\lambda(v_i); \bar{p})$ is satisfiable. We show that if φ is satisfiable, then it is satisfiable with a valuation for $\bar{p}$ with words of logarithmic size.

Consider the DFA A_i over $\Sigma_{\perp}^{m+m'}$ corresponding to the relation denoted by $\psi_i(\bar{x}; \bar{p})$, and observe that the number of such DFA as well as its sizes depend only on $\mathcal{A}$. Suppose there is a parameter valuation $v : \bar{p} \rightarrow \Sigma^*$ which makes φ true. This means that there is an accepting run of A_i on $w = v(p_1) \otimes \cdots \otimes v(p_m) \otimes \lambda(v_i)(x_1) \otimes \cdots \otimes \lambda(v_i)(x_{m'}) \in (\Sigma_{\perp}^{m+m'})^*$ for every i . Let q_i be the state of A_i in the accepting run after reading the N -prefix of w , and let A'_i be the same automaton as A_i but setting q_i as initial state. It then follows that, if w' is the N -prefix of w and w'' is any word in the language of A'_i , then $w' \cdot w''$ is in the language of A_i . Let $\pi(A'_i)$ be the automaton corresponding to the projection onto the last m components (i.e., the ones corresponding to the parameters) of $L(A'_i) \cap \{(a_1, \dots, a_{m'+m}) \in \Sigma_{\perp}^{m+m'} : a_1 = \cdots = a_{m'} = \perp\}^*$. Consider now any word $u \in \bigcap_i L(\pi(A'_i)) \subseteq (\Sigma_{\perp}^m)^*$, and observe that the description of $\bigcap_i L(\pi(A'_i))$ is bounded by a function of $\mathcal{A}$. Hence we can take u to be bounded by a constant. It then follows that $u' \cdot u$ witnesses also a satisfying assignment for φ , where u' is the N -prefix of $v(p_1) \otimes \cdots \otimes v(p_m)$. Hence, we can assume that the parameter valuation for witnessing a word in the language is also logarithmic in the size of G .

We can now simply guess all log-sized parameter valuations and traverse the crossproduct graph of $G \times \mathcal{A}$ using only logarithmic space. $\square$

Finally, observe that for the cases of $\mathbb{L}_{substr}$ and $\exists \mathbb{Z}_{LA}^{div}$ the log-size assumption make no difference, since in fact the reductions used for the proofs above already work in this restricted setting (see Appendix).

Corollary 4.7 (of the proof of Theorem 4.3). *Data complexity of $\mathbb{L}_{substr}$ -RPQ is NP-hard under the log-size assumption.*

Corollary 4.8 (of the proof of Theorem 4.2). *Data complexity of $\exists \mathbb{Z}_{LA}^{div}$ -RPQ is NP-hard under the log-size assumption.*

5 Extensions

5.1 $\mathbb{L}$ -CRPQ

It is easy to extend our results from $\mathbb{L}$ -RPQ, when the query additionally allows conjunctions, i.e., to the setting of Conjunctive Regular Path Queries (CRPQ) [16, 21, 24]. A query in $\mathbb{L}$ -CRPQ is of the form $Ans(\bar{z}) \leftarrow \bigwedge_{i=1}^m x_i \rightarrow_{L_i} y_i$, where $\bar{z} \subseteq \bar{x} \cup \bar{y}$ and each variable in $\bar{x} \cup \bar{y}$ is a node variable (like in RPQ), and each L_i is an $\mathbb{L}$ -pNFA. Given a typed data graph $G = (V, E, \lambda)$ and a vector $\bar{v}$ of nodes in V with $|\bar{v}| = |\bar{z}|$, we

write $G \models \text{Ans}(\bar{v})$ to mean that there exist a node valuation $\mu : (\bar{x} \cup \bar{y}) \rightarrow V$ that is consistent with the valuation to $\bar{z}$ (i.e. $\mu(z_j) = v_j$ for each $j \in [1, |\bar{z}|]$) such that each $\mathbb{L}$ -RPQ query $x_i \rightarrow_{L_i} y_i$ evaluates to true with start (resp. final) nodes $\mu(x_i)$ (resp. $\mu(y_i)$). Precisely the same results in Section 3 and in Section 4 trivially hold for $\mathbb{L}$ -CRPQ. This is because with a nondeterministic machine, one can first guess the node valuation μ and then invoke the algorithm for $\mathbb{L}$ -RPQ. Complexities like PSPACE are inherited here because NPSPACE coincides with PSPACE.

5.2 Mixing theories

It is worth remarking that the theories we have considered can be easily combined by means of boolean combination while keeping the complexity upper bounds. Consider, for instance, the theory $\mathbb{Z}_{LA}$ of linear integer arithmetic and $\mathbb{L}_{Aut}$ of string theories. Variables are either of type string or integer, and the schema contains both kind of types. Consider formulas which are Boolean combinations of formulas in the theory of strings (using string variables) and in the theory of linear arithmetic (using integer variables). Let us call $\mathbb{L}$ the resulting theory. We then have:

Theorem 5.1. *The data complexity of $\mathbb{L}$ -RPQ evaluation is in NL under the log-size assumption.*

Proof. Indeed, both upper bounds from Theorems 4.6 and 4.1 follow the same strategy, by finding a path on the product graph, and it is easy to see that they can be combined into one NL algorithm. $\square$

Example 4 from §2.5 can be easily expressed in $\mathbb{L}$ -RPQ.

5.3 Queries with aggregations

We show briefly how some of our results can be extended with aggregations. Example 5 in Section 2.5 – where the average age is constrained – is one query that can be expressed in our current extension.

In the sequel, we consider an extension of $\mathbb{Z}_{LA}$ -RPQ with aggregations. For this purpose, we introduce a concept of $\mathbb{Z}_{LA}$ -pNFA with costs (abbreviated as $\mathbb{Z}_{LA}$ -pNFAC). Henceforth we fix a schema $S = \{x_1, \dots, x_l\}$. The main idea of $\mathbb{Z}_{LA}$ -pNFAC is as follows:

- add into a $\mathbb{Z}_{LA}$ -pNFA a collection of cost variables, say $\bar{y}$, which range over the set of integers,
- transitions of the form $(q, (a, \psi), q')$ are adapted to $(q, (a, \psi, \alpha), q')$, where α assigns to each y_i a linear arithmetic expression for cost-variable updates: $\alpha(y_i) = \sum_{j=1}^m c_j y_j + d_0 + \sum_{j=1}^l d_j x_j$ with each $c_s, d_r \in \mathbb{Z}$,
- an initial value function I is added assigning to each y_i a value from $\mathbb{Z}$, and
- F is adapted to an accepting constraint, which is a partial function from Q to the set of linear arithmetic formulas $\mathbb{Z}_{LA}$ over $\bar{y}$.

In summary, a $\mathbb{Z}_{LA}$ -pNFAC $\mathcal{A}$ is a tuple $(R, I, Q, q_0, F, \Delta)$. A $\mathbb{Z}_{LA}$ -pNFAC is called *copyless* if $\alpha(y_i)$ is of the form $y_i + d_0 + \sum_{j'=1}^l d_{j'} x_{j'}$ in every transition. The language $\mathcal{L}(\mathcal{A})$ recognized by $\mathcal{A}$ is defined similarly as for $\mathbb{Z}_{LA}$ -pNFA, with the additional constraint that the final values of the cost variables satisfy the accepting constraint $F(q)$, with q being the last state in the run. (See Appendix E for a more formal definition of $\mathbb{Z}_{LA}$ -pNFAC.)

For instance, Example 5 in Section 2.5 can be captured by a copyless- $\mathbb{Z}_{LA}$ -pNFAC where two cost variables y_1, y_2 are used, for the sum of ages and the number of people respectively, and in each transition, $\alpha(y_1) = y_1 + x_2$ and $\alpha(y_2) = y_2 + 1$ (suppose that x_2 represents the age attribute), moreover, the accepting constraint is $y_1 < 40y_2$.

We define $\mathbb{Z}_{LA}$ -RPQC to be M -RPQ where $M = \mathbb{Z}_{LA}$ -pNFAC. Similarly, we define copyless- $\mathbb{Z}_{LA}$ -RPQC to be M -RPQ where $M = \text{copyless-}\mathbb{Z}_{LA}$ -pNFAC.

Proposition 5.2. *$\mathbb{Z}_{LA}$ -RPQC query evaluation is undecidable.*

In contrast, if we consider copyless- $\mathbb{Z}_{LA}$ -RPQC, then we may obtain positive results on query evaluation problem.

Theorem 5.3. *For the copyless- $\mathbb{Z}_{LA}$ -RPQC query evaluation problem, the data complexity is NP-complete, and NL-complete under log-size assumption. The combined complexity is in NEXP and NP-hard,*

6 Conclusion and Future Work

We initiated the study of path query languages over typed data graphs, where the power of SMT has been incorporated into the path constraint by means of $\mathbb{L}$ -pNFA for a decidable theory $\mathbb{L}$. Most importantly, we have shown the possibility of obtaining NL data complexity over some important logical theories including the theory of integer/rational linear arithmetic and the theory of automatic relations, in some cases having to impose a practically reasonable log-size data assumption. Despite this, we have also shown the impossibility of obtaining NL algorithms for some logical theories (e.g. existential Presburger Arithmetic with divisibility [37] and the existential theory of strings with the substring-of relation [35]).

We believe there are several interesting avenues for future work. Firstly, it would be interesting to further study the possibility of obtaining NL algorithms for other theories, e.g., Tarski's Real Field, which has played an important role in constraint databases [34]. We plan to also explore other extensions including query languages like (e.g. GX-Path [38], ECRPQ [6]) and more powerful aggregations. On the practical side, it would be interesting to see if our NL algorithms could be incorporated into some existing graph query evaluation system (e.g. [4]). To this end, one could instead utilize a fast SMT-solver (like Z3 [18]) for solving the final satisfiability query in the theory.

References

- [1] JCC Consulting Inc. acting on behalf of an unincorporated association of ISO Graph Query Language Proponents. [n.d.]. *GQL Standard Website*. <https://www.gqlstandards.org/>
- [2] Renzo Angles and Claudio Gutiérrez. 2008. Survey of graph database models. *ACM Comput. Surv.* 40, 1 (2008), 1:1–1:39. <https://doi.org/10.1145/1322432.1322433>
- [3] Pablo Barceló Baeza. 2013. Querying graph databases. In *Proceedings of the 32nd ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems, PODS 2013, New York, NY, USA - June 22 - 27, 2013*, Richard Hull and Wenfei Fan (Eds.). ACM, 175–188. <https://doi.org/10.1145/2463664.2465216>
- [4] Jorge Baier, Dietrich Daroch, Juan L. Reutter, and Domagoj Vrgoc. 2017. Evaluating Navigational RDF Queries over the Web. In *Proceedings of the 28th ACM Conference on Hypertext and Social Media, HT 2017, Prague, Czech Republic, July 4-7, 2017*, Peter Dolog, Peter Vojtás, Francesco Bonchi, and Denis Helic (Eds.). ACM, 165–174. <https://doi.org/10.1145/3078714.3078731>
- [5] Pablo Barceló, Gaëlle Fontaine, and Anthony Widjaja Lin. 2015. Expressive Path Queries on Graph with Data. *Log. Methods Comput. Sci.* 11, 4 (2015). [https://doi.org/10.2168/LMCS-11\(4:1\)2015](https://doi.org/10.2168/LMCS-11(4:1)2015)
- [6] Pablo Barceló, Leonid Libkin, Anthony Widjaja Lin, and Peter T. Wood. 2012. Expressive Languages for Path Queries over Graph-Structured Data. *ACM Trans. Database Syst.* 37, 4 (2012), 31:1–31:46. <https://doi.org/10.1145/2389241.2389250>
- [7] Michael Benedikt, Leonid Libkin, Thomas Schwentick, and Luc Segoufin. 2003. Definable relations and first-order query languages over strings. *J. ACM* 50, 5 (2003), 694–751. <https://doi.org/10.1145/876638.876642>
- [8] Henrik Björklund and Mikolaj Bojanczyk. 2007. Shuffle Expressions and Words with Nested Data. In *Mathematical Foundations of Computer Science 2007, 32nd International Symposium, MFCS 2007, Český Krumlov, Czech Republic, August 26-31, 2007, Proceedings (Lecture Notes in Computer Science, Vol. 4708)*, Ludek Kucera and Antonín Kucera (Eds.). Springer, 750–761. https://doi.org/10.1007/978-3-540-74456-6_66
- [9] Michael Blondin, Alain Finkel, Stefan Göller, Christoph Haase, and Pierre McKenzie. 2015. Reachability in Two-Dimensional Vector Addition Systems with States Is PSPACE-Complete. In *30th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2015, Kyoto, Japan, July 6-10, 2015*. IEEE Computer Society, 32–43.
- [10] Achim Blumensath and Erich Grädel. 2000. Automatic structures. In *Logic in Computer Science, 2000. Proceedings. 15th Annual IEEE Symposium on*. IEEE, 51–62.
- [11] Achim Blumensath and Erich Grädel. 2000. Automatic Structures. In *15th Annual IEEE Symposium on Logic in Computer Science, Santa Barbara, California, USA, June 26-29, 2000*. IEEE Computer Society, 51–62. <https://doi.org/10.1109/LICS.2000.855755>
- [12] Achim Blumensath and Erich Grädel. 2004. Finite presentations of infinite structures: Automata and interpretations. *Theory of Computing Systems* 37, 6 (2004), 641–674.
- [13] Bernard Boigelot, Stéphane Rassart, and Pierre Wolper. 1998. On the Expressiveness of Real and Integer Arithmetic Automata (Extended Abstract). In *Automata, Languages and Programming, 25th International Colloquium, ICALP '98, Aalborg, Denmark, July 13-17, 1998, Proceedings (Lecture Notes in Computer Science, Vol. 1443)*, Kim Guldstrand Larsen, Sven Skyum, and Glynn Winskel (Eds.). Springer, 152–163. <https://doi.org/10.1007/BFb0055049>
- [14] Mikolaj Bojanczyk, Claire David, Anca Muscholl, Thomas Schwentick, and Luc Segoufin. 2011. Two-variable logic on data words. 12, 4 (2011), 27:1–27:26. <https://doi.org/10.1145/1970398.1970403>
- [15] Véronique Bruyere, Georges Hansel, Christian Michaux, and Roger Villemaire. 1994. Logic and p-recognizable sets of integers. *Bulletin of the Belgian Mathematical Society Simon Stevin* 1, 2 (1994), 191–238.
- [16] Diego Calvanese, Giuseppe De Giacomo, Maurizio Lenzerini, and Moshe Y. Vardi. 2000. Containment of Conjunctive Regular Path Queries with Inverse. In *KR 2000, Principles of Knowledge Representation and Reasoning Proceedings of the Seventh International Conference, Breckenridge, Colorado, USA, April 11-15, 2000*, Anthony G. Cohn, Fausto Giunchiglia, and Bart Selman (Eds.). Morgan Kaufmann, 176–185.
- [17] Loris D'Antoni and Margus Veanes. 2021. Automata modulo theories. *Commun. ACM* 64, 5 (2021), 86–95. <https://doi.org/10.1145/3419404>
- [18] L. Mendonça de Moura and N. Bjørner. 2008. Z3: An Efficient SMT Solver. In *TACAS*.
- [19] Normann Decker, Peter Habermehl, Martin Leucker, and Daniel Thoma. 2014. Ordered Navigation on Multi-attributed Data Words. In *CONCUR 2014 - Concurrency Theory - 25th International Conference, CONCUR 2014, Rome, Italy, September 2-5, 2014. Proceedings (Lecture Notes in Computer Science, Vol. 8704)*, Paolo Baldan and Daniele Gorla (Eds.). Springer, 497–511. https://doi.org/10.1007/978-3-662-44584-6_34
- [20] Normann Decker and Daniel Thoma. 2016. On Freeze LTL with Ordered Attributes. In *Foundations of Software Science and Computation Structures - 19th International Conference, FOSSACS 2016, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2016, Eindhoven, The Netherlands, April 2-8, 2016, Proceedings (Lecture Notes in Computer Science, Vol. 9634)*, Bart Jacobs and Christof Löding (Eds.). Springer, 269–284. https://doi.org/10.1007/978-3-662-49630-5_16
- [21] Alin Deutsch and Val Tannen. 2001. Optimization Properties for Classes of Conjunctive Regular Path Queries. In *Database Programming Languages, 8th International Workshop, DBPL 2001, Frascati, Italy, September 8-10, 2001, Revised Papers (Lecture Notes in Computer Science, Vol. 2397)*, Giorgio Ghelli and Gösta Grahne (Eds.). Springer, 21–39. https://doi.org/10.1007/3-540-46093-4_2
- [22] Herbert B. Enderton. 2001. *Introduction to Mathematical Logic* (2 ed.). Academic Press.
- [23] Wenfei Fan. 2015. Data Quality: From Theory to Practice. *SIGMOD Rec.* 44, 3 (2015), 7–18. <https://doi.org/10.1145/2854006.2854008>
- [24] Daniela Florescu, Alon Y. Levy, and Dan Suciu. 1998. Query Containment for Conjunctive Queries with Regular Expressions. In *Proceedings of the Seventeenth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems, June 1-3, 1998, Seattle, Washington, USA*, Alberto O. Mendelzon and Jan Paredaens (Eds.). ACM Press, 139–148. <https://doi.org/10.1145/275487.275503>
- [25] Christiane Frougny and Jacques Sakarovitch. 1991. Rational Relations with Bounded Delay. In *STACS 91, 8th Annual Symposium on Theoretical Aspects of Computer Science, Hamburg, Germany, February 14-16, 1991, Proceedings (Lecture Notes in Computer Science, Vol. 480)*, Christian Choffrut and Matthias Jantzen (Eds.). Springer, 50–63. <https://doi.org/10.1007/BFb0020787>
- [26] M. R. Garey and David S. Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman.
- [27] Christoph Haase. 2018. A survival guide to presburger arithmetic. *ACM SIGLOG News* 5, 3 (2018), 67–82. <https://dl.acm.org/citation.cfm?id=3242964>
- [28] Christoph Haase and Simon Halfon. 2014. Integer Vector Addition Systems with States. In *Reachability Problems - 8th International Workshop, RP 2014, Oxford, UK, September 22-24, 2014. Proceedings (Lecture Notes in Computer Science, Vol. 8762)*, Joël Ouaknine, Igor Potapov, and James Worrell (Eds.). Springer, 112–124.
- [29] Jelle Hellings, Bart Kuijpers, Jan Van den Bussche, and Xiaowang Zhang. 2013. Walk logic as a framework for path query languages on graph databases. In *Joint 2013 EDBT/ICDT Conferences, ICDT '13 Proceedings, Genoa, Italy, March 18-22, 2013*, Wang-Chiew Tan, Giovanna Guerrini, Barbara Catania, and Anastasios Gounaris (Eds.). ACM, 117–128. <https://doi.org/10.1145/2448496.2448512>

- [30] Michael Kaminski and Nissim Francez. 1994. Finite-Memory Automata. 134, 2 (1994), 329–363. [https://doi.org/10.1016/0304-3975\(94\)90242-9](https://doi.org/10.1016/0304-3975(94)90242-9)
- [31] Dexter Kozen. 1977. Lower Bounds for Natural Proof Systems. In *18th Annual Symposium on Foundations of Computer Science, Providence, Rhode Island, USA, 31 October - 1 November 1977*. IEEE Computer Society, 254–266. <https://doi.org/10.1109/SFCS.1977.16>
- [32] Dexter C. Kozen. 2006. *Theory of Computation*. Springer.
- [33] Daniel Kroening and Ofer Strichman. 2008. *Decision Procedures*. Springer.
- [34] Gabriel Kuper, Leonid Libkin, and Jan Paredaens. 2000. *Constraint Databases*. Springer.
- [35] Dietrich Kuske. 2006. Theories of orders on the set of words. *RAIRO Theor. Informatics Appl.* 40, 1 (2006), 53–74. <https://doi.org/10.1051/ita:2005039>
- [36] Kevin R. Lawrence. 2021. *Practical Gremlin: An Apache TinkerPop Tutorial*. <https://kelvinlawrence.net/book/Gremlin-Graph-Guide.html>
- [37] Antonia Lechner, Joël Ouaknine, and James Worrell. 2015. On the Complexity of Linear Arithmetic with Divisibility. In *30th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2015, Kyoto, Japan, July 6–10, 2015*. IEEE Computer Society, 667–676. <https://doi.org/10.1109/LICS.2015.67>
- [38] Leonid Libkin, Wim Martens, and Domagoj Vrgoc. 2016. Querying Graphs with Data. *J. ACM* 63, 2 (2016), 14:1–14:53. <https://doi.org/10.1145/2850413>
- [39] Leonid Libkin and Domagoj Vrgoc. 2012. Regular path queries on graphs with data. In *15th International Conference on Database Theory, ICDT '12, Berlin, Germany, March 26–29, 2012*, Alin Deutsch (Ed.). ACM, 74–85. <https://doi.org/10.1145/2274576.2274585>
- [40] Fabio Massacci and Ida Sri Rejeki Siahaan. 2008. Simulating midlet's security claims with automata modulo theory. In *Proceedings of the 2008 Workshop on Programming Languages and Analysis for Security, PLAS 2008, Tucson, AZ, USA, June 8, 2008*, Úlfar Erlingsson and Marco Pistoia (Eds.). ACM, 1–9. <https://doi.org/10.1145/1375696.1375698>
- [41] Fabio Massacci and Ida Siahaan. 2009. *Automata Modulo Theories (AMT)*. Technical Report. DISI, University of Trento.
- [42] Alberto O. Mendelzon and Peter T. Wood. 1995. Finding Regular Simple Paths in Graph Databases. *SIAM J. Comput.* 24, 6 (1995), 1235–1258. <https://doi.org/10.1137/S009753979122370X>
- [43] Neo4j. [n.d.]. *Neo4j's Developer Guides to Cypher Query Language*. <https://neo4j.com/developer/cypher/>
- [44] Julia Robinson. 1949. Definability and decision problems in arithmetic. *The Journal of Symbolic Logic* 14, 2 (1949), 98–114.
- [45] Peter T. Wood. 2012. Query languages for graph databases. *SIGMOD Rec.* 41, 1 (2012), 50–60. <https://doi.org/10.1145/2206869.2206879>

A Proof of Theorem 4.2

Proof of Theorem 4.2. The reduction is from 3-SAT. The fixed automaton $\mathcal{A}$ has one state q , which is both initial and final, and it has two transitions $\{(q, (\dagger, \psi(x; p)), q) : \dagger \in \{a, \#\}\}$ for some fixed $a \in \Sigma$, where

$\psi(x; p) = 2 \dagger p \wedge 4 \dagger x \wedge \exists y ((x = 2y \wedge y \mid p) \vee (2 \dagger x \wedge x \dagger p))$ and p is the parameter. Suppose the given 3-SAT formula is

$$\varphi := \bigwedge_{i=1}^m (\ell_{i,1} \vee \ell_{i,2} \vee \ell_{i,3})$$

over the variables $x_1, \dots, x_n$. The data graph is $G = (V, E)$, where $V = \{v_{i,j} : i \in [1, m], j \in \{1, 2, 3\}\} \cup \{v_0, v_F\}$, and

$$E = \{(v_0, a, v_{1,j}), (v_{m,j}, a, v_F) : j \in \{1, 2, 3\}\} \cup \{(v_{i,j}, a, v_{i+1,j'}) : i \in [1, n-1], j, j' \in \{1, 2, 3\}\}.$$

We set $\lambda(v_0) = \lambda(v_F) = 2$. In addition, we set:

- if $\ell_{i,j} = x_k$, set $\lambda(v_{i,j}) = 2 \times P_{k+1}$, where P_k is the k th prime.
- if $\ell_{i,j} = \neg x_k$, set $\lambda(v_{i,j}) = P_{k+1}$, where P_k is the k th prime.

If φ is satisfiable by setting variables $x_{i_1}, \dots, x_{i_r}$ to true and the remaining ones to false, it follows that there is a path from v_0 to v_F in the language of $\mathcal{A}$ passing through all true clauses, using the parameter assignment $p \mapsto \prod_{j=1}^r P_{i_j+1}$. Conversely, if there is a path from v_0 to v_F in the language with a parameter assignment $z \in \mathbb{N}$, the formula φ is satisfiable by setting every x_j to true if P_{j+1} divides z , or to false otherwise. $\square$

B Proof of Theorem 4.3

Proof of Theorem 4.3. We reduce from 3-SAT. The automaton has just one state q , both initial and final, one parameter p , and two transitions $\{(q, (\dagger, \psi(x_1, x_2; p)), q) : \dagger \in \{\#, a\}\}$, with

$$\psi(x_1, x_2; p) = x_1 \leq_{\text{substr}} p \wedge \neg(x_2 \leq_{\text{substr}} p)$$

Given a 3-SAT formula

$$\varphi := \bigwedge_{i=1}^m (\ell_{i,1} \vee \ell_{i,2} \vee \ell_{i,3})$$

over n variables $z_1, \dots, z_n$, consider the graph over $\Sigma = \{a\}$ having vertices $V = \{v_{i,j} : i \in [1, m], j \in \{1, 2, 3\}\} \cup \{v_0, v_F\}$ and edges

$$E = \{(v_0, a, v_{1,j}), (v_{m,j}, a, v_F) : j \in \{1, 2, 3\}\} \cup \{(v_{i,j}, a, v_{i+1,j'}) : i \in [1, n-1], j, j' \in \{1, 2, 3\}\}.$$

For any $k \in \{1, \dots, n\}$, let $w_k \in \{0, 1\}^*$ be the binary encoding of the number k into $\lceil \log(n) \rceil$ bits. We set $\lambda(v_0)(x_1) = \lambda(v_F)(x_1) = \varepsilon$ and $\lambda(v_0)(x_2) = \lambda(v_F)(x_2) = \$$. For any other $v_{i,j}$ such that $\ell_{i,j} = z_k$ (resp. $\ell_{i,j} = \neg z_k$), we set $\lambda(v_{i,j})$ to be the following strings over $\{0, 1, \%\}$ of length $\lceil \log(n) \rceil + 3$:

$$\begin{aligned} \lambda(v_{i,j})(x_1) &= \% w_k 1 \% \quad (\text{resp. } \lambda(v_{i,j})(x_1) = \% w_k 0 \%) \\ \lambda(v_{i,j})(x_2) &= \% w_k 0 \% \quad (\text{resp. } \lambda(v_{i,j})(x_2) = \% w_k 1 \%). \end{aligned}$$

It then follows that a satisfying assignment produces a path and a parameter value that makes (v_0, v_F) in the query result, and conversely any path from v_0 to v_F with reading a path in the language yields a satisfying assignment in the witnessing parameter value. $\square$

C Proof of Theorem 4.4

Proof of Theorem 4.4. We reduce from 3-SAT. The automaton has just one state q , both initial and final, one parameter p , and two transitions $\{(q, (\dagger, \psi(x; p)), q) : \dagger \in \{\#, a\}\}$, with

$$\psi(x; p) = \exists p', x' \ x' < x \wedge p' < p \wedge \text{eq-len}(p', x') \wedge \text{lst}_1(x') \wedge (\text{lst}_1(p') \leftrightarrow \text{lst}_1(x))$$

Given a 3-SAT formula

$$\varphi := \bigwedge_{i=1}^m (\ell_{i,1} \vee \ell_{i,2} \vee \ell_{i,3})$$

Consider the graph over $\Sigma = \{a\}$ having vertices $V = \{v_{i,j} : i \in [1, m], j \in \{1, 2, 3\}\} \cup \{v_0, v_F\}$ and edges

$$E = \{(v_0, a, v_{1,j}), (v_{m,j}, a, v_F) : j \in \{1, 2, 3\}\} \cup \{(v_{i,j}, a, v_{i+1,j'}) : i \in [1, n-1], j, j' \in \{1, 2, 3\}\}.$$

We set $\lambda(v_0) = \lambda(v_F) = \varepsilon$. For any other $v_{i,j}$ such that $\ell_{i,j} = x_k$ (resp. $\ell_{i,j} = \neg x_k$) we set $\lambda(v_{i,j})$ to be the string over $\{0, 1\}$ of length $m+1$ such that:

- its k -th letter is 1,
- its last letter is 1 (resp. 0), and
- every other letter is 0.

If φ is satisfiable via the truth assignment $x_{i_1}, \dots, x_{i_r}$, then there is a path from v_0 to v_F in the language with the parameter assignment $\{0, 1\}^{m+1}$ which has a 1-symbol on the i_j -th position, for every $1 \leq j \leq r$, and a 0-symbol otherwise. Conversely, for any path from v_0 to v_F in the language, the m -prefix of the parameter assignment witnessing membership yields a satisfying assignment for φ . $\square$

D Proof of Theorem 4.5

Proof of Theorem 4.5. Let $\mathcal{A} = (R, Q, q_0, F, \Delta)$ be a weak $\mathbb{L}_{Aut}$ -pNFA. For simplicity let us assume that Σ is both the alphabet of the data graph and of the string theory.

The NL algorithm has one string $w_\xi \in \Sigma^* \cup \{?\}$ for every atomic formula of $\mathcal{A}$, which is initialized in $?$, meaning “undefined”. It also keeps formulas $F(\bar{p})$ and $F'(\bar{p})$ (initially $F' = F = \top$). The size of F' will stay bounded by the size of the automaton (hence constant), and F will be a conjunction of (negated) atoms of size logarithmic in G . As before, the algorithm traverses the product graph of G and $\mathcal{A}$. We start at (s, q_0) , where s is the source vertex and q_0 the initial state. At each step, assuming that we are at (v, q) , we guess a vertex (v', q') , a transition $(q, (a, \varphi(\bar{x}; \bar{p})), q')$ and a disjunct $\psi(\bar{p}) \wedge \psi'(\bar{x};) \wedge \xi_1 \wedge \dots \wedge \xi_m$ of φ of the form (2), such that $v \xrightarrow{a} v'$ is an edge of G . We first check that the valuation of $\lambda(v)$ verifies $\psi'(\bar{x};)$, otherwise we reject the computation. Next, we update F and each w_{ξ_i} as follows:

- if $\xi_i = p \leq x$, and $w_{\xi_i} = ?$, we set $w_{\xi_i} := \lambda(v)(x)$;
- if $\xi_i = p \leq x$, and $w_{\xi_i} \neq ?$, we set w_{ξ_i} to be the largest common prefix between w_{ξ_i} and $\lambda(v)(x)$;
- if $\xi_i = p \geq x$, and $w_{\xi_i} = ?$, we set $w_{\xi_i} := \lambda(v)(x)$;
- if $\xi_i = p \geq x$, and $w_{\xi_i} \neq ?$, we check that w_{ξ_i} and $\lambda(v)(x)$ are prefix-comparable, and we set w_{ξ_i} to be the largest one;
- if $\xi_i = p \sim x$ for an equivalence relation $\sim$, we first test if F contains a conjunct of the form $p \sim w$ for some s . If so, we check $w' \sim w$ for $w' = \lambda(v)(x)$ in NL. If it does not contain such a conjunct, we set $F := F \wedge p \sim w'$.

Observe that in the construction of F we don't need to write down the concrete values of $\lambda(v)(x)$, but we can simply put a pointer to the string value, and hence each of the verifications is in NL. If any of the verifications above fails,

the computation is aborted. Finally, if $\psi(\bar{p})$ is not a conjunct of F' , we set $F' := F' \wedge \psi$. The algorithm continues until it reaches a vertex (t, q_f) where t is the target vertex and q_f is a final state. Finally, it verifies that $\varphi = (R \wedge F' \wedge F)(\bar{p})$ is satisfiable. This last satisfiability checking can be done in NL since R and F' are of constant size and F is a conjunction of atoms of the form $p \leq w, p \geq w$ and $p \sim w$ for some (prefixes of) words w in the database, which can all be verified in NL. More concretely, let us assume $\bar{p} = p_1, \dots, p_k$ and let $\sim_1, \dots, \sim_\ell$ be the automatic partial equivalence relations in F , and let $F = \xi_1 \wedge \dots \wedge \xi_m$. One way of testing satisfiability is first computing (in constant time) a DFA over $\Sigma_\perp^m$, as the convolution of the automatic relation defined by

$$\mu(\bar{z}) = \exists p_1, \dots, p_k R \wedge F' \wedge \bigwedge_{\xi=(w \geq p_i)} z_\xi \geq p_i \wedge \bigwedge_{\xi=(w \leq p_i)} z_\xi \leq p_i \wedge \bigwedge_{\xi=(w \sim_j p_i)} z_\xi \sim_j p_i$$

where $\bar{z} = z_{\xi_1}, \dots, z_{\xi_m}$. Observe that this depends only on the automaton and hence can be computed in constant time. Now we test that for each of the words w_i in each ξ_i , that $\mu[w_1, \dots, w_m]$ holds true. This can be done by traversing the transition graph of the DFA from the initial state and reading $w_1 \otimes \dots \otimes w_m$, which can be done in logarithmic space, and finally checking that we arrive to a final state.

Observe that if $\varphi(\bar{p})$ is satisfiable, the witnessing parameter values yields an accepting run for the path between s and t that was traversed. Conversely, for any satisfying path the parameter values needed to accept the word is a satisfying assignment for φ . $\square$

E $\mathbb{Z}_{LA}$ -RPQ with costs

Definition E.1 ($\mathbb{Z}_{LA}$ -pNFA). A $\mathbb{Z}_{LA}$ -pNFA with costs (abbreviated as $\mathbb{Z}_{LA}$ -pNFA) with parameters $\bar{p}$ and cost variables $\bar{y} = (y_1, \dots, y_m)$ on a schema $S = (x_1, \dots, x_l)$ is a tuple $\mathcal{A} = (R, I, Q, q_0, F, \Delta)$, where R, Q, q_0 are as in $\mathbb{Z}_{LA}$ -pNFA, $I : \bar{y} \rightarrow \mathbb{Z}$ is a function that assigns to each y_i an initial value, Δ is a finite set of transitions $(q, (a, \psi, \alpha), q')$ such that $q, q' \in Q, a \in \Sigma, \psi \in \mathbb{Z}_{LA}$, and α assigns to each y_i a linear combination expression $\alpha(y_i) = \sum_{j=1}^m c_j y_j + d_0 + \sum_{j'=1}^l d_{j'} x_{j'}$ with $c_1, \dots, c_m, d_0, d_1, \dots, d_l \in \mathbb{Z}$. Finally, F specifies the accepting constraints, which is a partial function from Q to the set of linear arithmetic formulas $\mathbb{Z}_{LA}$ over $\bar{y}$. Intuitively, an accepting run is one that ends in a state q which is in the domain of F and such that the formula $F(q)$ is satisfied with the current values of $\bar{y}$. A $\mathbb{Z}_{LA}$ -pNFA is called *copyless* if $\alpha(y_i)$ is of the form $y_i + d_0 + \sum_{j'=1}^l d_{j'} x_{j'}$ in every transition.

The language $\mathcal{L}(\mathcal{A})$ recognized by $\mathcal{A}$ is defined similarly as for $\mathbb{Z}_{LA}$ -pNFA. Take a path

$$\pi : q_0 \rightarrow_{(a_1, \psi_1(\bar{x}_1; \bar{p}), \alpha_1)} \dots \rightarrow_{(a_n, \psi_n(\bar{x}_n; \bar{p}), \alpha_n)} q_n$$

in $\mathcal{A}$ such that $F(q_n)$ is defined. By relabeling the non-parametric variables in $\bar{x}_1, \dots, \bar{x}_n$, we assume in the following that $\bar{x}_i$ and $\bar{x}_j$ are disjoint, for each $i \neq j$. Suppose there is a valuation v such that $\mathfrak{S}, v \models \exists \bar{p}(\varphi(\bar{p}) \wedge \bigwedge_{i=1}^n \psi(\bar{x}_i; \bar{p}))$, moreover, $F(q_n)(\bar{\beta}_n)$ is evaluated to true, where $\bar{\beta}_0 = I(\bar{y})$ and $\bar{\beta}_i = \alpha_i(\bar{y})(\bar{\beta}_{i-1}, v(\bar{x}_i))$ for $i \in [n]$, then we add the word $(a_1, v(\bar{x}_1)) \cdots (a_n, v(\bar{x}_n))$ to $\mathcal{L}(\mathcal{A})$.

Theorem 5.3. *For the copyless- $\mathbb{Z}_{LA}$ -RPQC query evaluation problem, the combined complexity is in NEXP and NP-hard, the data complexity is NP-complete, and NL-complete under log-size assumption.*

Proof. We first show the NP-hardness of the data complexity. We get a reduction from the subset sum problem [26]. In subset sum problem, we are given a finite set A , size $s(a) \in \mathbb{Z}$ for each $a \in A$, and a positive integer B , and decide whether there is a subset $A' \subseteq A$ such that the sum of the sizes of the elements in A' is exactly B .

Let $A = \{a_1, \dots, a_n\}$, $\Sigma = \{*, \$\}$, and $S = \{x_1\}$. The reduction is obtained by constructing an instance of the copyless- $\mathbb{Z}_{LA}$ -RPQC query evaluation problem, namely (G, s, t) and the copyless- $\mathbb{Z}_{LA}$ -pNFAC $\mathcal{A}$, as follows:

- the data graph $G = (V, E, \lambda)$, where $V = A \cup \{a_0, a_{n+1}, a_f\}$, E comprises the tuples $(a_{i-1}, *, a_i)$ for $i \in [n+1]$, $(a_{n+1}, \$, a_f)$, moreover, $\lambda(a_0)(x_1) = 0$, $\lambda(a_i)(x_1) = s(a_i)$ for $i \in [n]$, $\lambda(a_{n+1})(x_1) = B$, and $\lambda(a_f)(x_1) = 0$,
- let $\bar{y} = (y_1)$, then $\mathcal{A} = (R, I, Q, q_0, F, \Delta)$, where
 - $R = \text{true}$, $I(y_1) = 0$,
 - $Q = \{q_0, q_f\}$,
 - Δ comprises the two transitions $(q_0, (*, \text{true}, \alpha_1), q_0)$ and $(q_0, (*, \text{true}, \alpha_2), q_0)$ with $\alpha_1(y_1) = y_1 + x_1$ and $\alpha_2(y_1) = y_1$ (intuitively, α_1 and α_2 correspond to the situation that an item a_i is chosen resp. not chosen in the subset), as well as $(q_0, (\$, \text{true}, \beta), q_f)$ with $\beta(y_1) = y_1 - x_1$,
 - $F(q_f)$ is $y_1 = 0$.
- $s = a_0$ and $t = a_f$.

From the construction, we know that a given instance of the subset sum problem returns true iff there is a path in G from a_0 to a_f such that the data word corresponding to the path is accepted by $\mathcal{A}$. It is easy to see that the reduction is a polynomial-time reduction.

Moreover, the size of the copyless- $\mathbb{Z}_{LA}$ -pNFAC in the aforementioned reduction is independent of the size of the subset sum problem instance, thus a constant. From this, we conclude that the data complexity of the copyless- $\mathbb{Z}_{LA}$ -RPQC query evaluation problem is NP-hard.

On the other hand, the NL-lower bound for the data complexity under log-size assumption follows from the fact that the $\mathbb{Z}_{LA}$ -RPQ query evaluation under log-size assumption is already NL-hard.

For the upper bounds on the combined and data complexity, we show them by a reduction to the query evaluation

problem to the reachability of an integer vector addition systems with states (VASS).

In the sequel, we assume that the data graph is $G = (V, E, \lambda)$ and the $\mathbb{L}_{\leq, +}$ -pNFAC with parameters $\bar{p}$ and cost registers $\bar{y} = (y_1, \dots, y_m)$ on a cost schema (S, f) with $S = (x_1, \dots, x_l)$ is $\mathcal{A} = (R, I, Q, q_0, F, \Delta)$. Moreover, let $s, t \in V$. W.l.o.g. we assume that for each $q \in Q$, $F(q)$ is of the form $\bigwedge_{i=1}^p (c_{i,0} + \sum_{j=1}^m c_{i,j} y_j) \sim_i 0$, where $\sim_i \in \{<, >, =, \equiv_d\}$.

Otherwise, $\mathcal{A}$ can be transformed into one satisfying this constraint by rewriting $F(q)$ into disjunctive normal form, splitting q into multiple copies, one for each disjunct, and adapting the transitions into q correspondingly.

Consider a path in the product of G and $\mathcal{A}$ from (s, q_0) to (t, q) for some $q \in Q$ such that $F(q)$ is satisfied by the final values of $\bar{y}$, say $v(\bar{y})$.

Let us fix some $q_f \in Q$ such that $F(q_f) = \bigwedge_{i=1}^p (c_{i,0} + \sum_{j=1}^m c_{i,j} y_j) \sim_i 0$. Then the integer VASS we are going to define contains vectors of p dimensions, to record the updates on the value of $c_{i,0} + \sum_{j=1}^m c_{i,j} y_j$ for $i \in [p]$.

Let $\mathcal{A}'$ be the $\mathbb{Z}_{LA}$ -pNFA obtained from $\mathcal{A}$ by removing the cost variables, namely, $\mathcal{A}' = (R, Q, q_0, F', \Delta')$, where $F' = \{q \in Q \mid F(q) \text{ is defined}\}$ and Δ' comprises the tuples $(q, (a, \psi), q')$ such that $(q, (a, \psi, \alpha), q') \in \Delta$ for some α .

From the proof of Theorem 4.1, the NL decision procedure for the $\mathbb{Z}_{LA}$ -pNFA $\mathcal{A}'$ can be understood in another way: Let $\text{Atoms}(\mathcal{A}')$ denote the set of atomic formulas $h(\bar{p}) \sim h'(\bar{x})$ in $\mathcal{A}'$. Then the evaluation of $\mathcal{A}'$ on (G, s, t) is reduced to the reachability in a graph $G' = (V', E')$ where $V' = V \times Q \times \mathcal{F}$ such that $\mathcal{F}$ is the set of tuples $((n_\psi)_{\psi \in \text{Atoms}(\mathcal{A}')} , \varphi)$. Note that since each n_ψ has polynomially many choices and φ is determined by $((n_\psi)_{\psi \in \text{Atoms}(\mathcal{A}')} , \varphi)$, moreover, $|\text{Atoms}(\mathcal{A}')|$ is a constant (since we are considering the data complexity), we deduce that $|V'|$ is at most polynomially in $|V|$.

Then we define the integer VASS $\mathcal{B} = (V', \Delta')$ where Δ' comprises the transitions $(v'_1, (e_1, \dots, e_p), v'_2)$ such that $v'_1 = (v_1, q_1, \theta_1)$, $v'_2 = (v_2, q_2, \theta_2)$, and there is a transition $(q_1, (a, \psi, \alpha), q_2) \in \Delta$ such that θ_2 is constructed from θ_1 and $(q_1, (a, \psi), q_2)$ in G' , moreover, for each $j : 1 \leq j \leq m$, let $\alpha(y_j) = y_j + d_{j,0} + \sum_{j'=1}^l d_{j,j'} x_{j'}$, then for each $i : 1 \leq i \leq p$,

$$e_i = \sum_{j=1}^m c_{i,j} \left(d_{j,0} + \sum_{j'=1}^l d_{j,j'} \lambda(v_2)(x_{j'}) \right).$$

Intuitively, e_i reflects the updates on $c_{i,0} + \sum_{j=1}^m c_{i,j} y_j$ brought upon by $(q_1, (a, \psi, \alpha), q_2) \in \Delta$.

Moreover, some auxiliary states are added into $\mathcal{B}$ to set the initial value of the i -th component of the vector as $c_{i,0}$, as well as dealing with $\sim_i \in \{<, >, \equiv_d\}$, where $1 \leq i \leq p$.

For instance, if $c_{i,0} = 1$ and $\sim_i$ is $<$, let 1_i denote the vector of length p where the i -th component is 1 and all the other components are zero, then an auxiliary fresh state q'_0 and a transition $(q'_0, 1_i, (s, q_0, \theta_0))$ are added (where $\theta_0 = ((n_\psi)_{\psi \in \text{Atoms}(\mathcal{A}')} , \varphi)$ satisfies that $n_\psi = ?$ and $\varphi = \top$), moreover, a fresh state q'_f and the transitions $((t, q_f, \theta), 1_i, q_f)$ and $(q'_f, 1_i, q'_f)$ are added. Then the query evaluation of $\mathcal{A}$ on (G, s, t) is successful iff $(q'_f, \bar{0})$ is reachable from $(q'_0, \bar{0})$ in $\mathcal{B}$.

If the combined complexity is concerned, $|V'|$ and $|\Delta'|$ are at most exponential in $|V|$. From the fact that the reachability of integer VASS is NP-complete [28], we conclude that the combined complexity of copyless- $\mathbb{Z}_{LA}$ -pNFAC is in NEXP.

Next, let us consider the data complexity.

In this case, $|V'|$ and $|\Delta'|$ are at most polynomial in $|V|$. Therefore, we obtain a reduction to the reachability problem of an integer VASS $\mathcal{B}$ of polynomial size (where the integer constants are encoded in binary). From the fact that the reachability of integer VASS is NP-complete [28], we conclude that the data complexity of copyless- $\mathbb{Z}_{LA}$ -pNFAC is in NP, thus NP-complete.

Next, let us consider the data complexity under the log-size assumption. According to the log-size assumption, each tuple $((n_\psi)_{\psi \in \text{Atoms}(\mathcal{A}')} , \varphi)$ in G' occupies logarithmic space, thus $|\mathcal{F}|$ is polynomial in the size of G , say bounded by $|V|^r$ for some constant r .

From the log-size assumption and the fact that the size of $\mathcal{A}$ is considered as a constant (since data complexity is considered), we know that the value of each e_i above is polynomial in the size of G , say bounded by $|V|^{r'}$ for some constant r' . Therefore, $|\Delta'|$, namely, the number of transitions in $\mathcal{B}$, is at most $|V|^r |V|^{r'p} |V|^r = |V|^{2r+r'p}$. In addition, the maximum absolute value of constants in Δ' , denoted by $||\Delta'|$, is at most $|V|^{r'}$.

Therefore, we can think $\mathcal{B}$ as a unary integer VASS of dimension p . From the proof of Theorem 19 in [9], we know that if $(q'_f, \bar{0})$ is reachable from $(q'_0, \bar{0})$ in $\mathcal{B}$, then this reachability can be witnessed by a path of length at most $|\Delta'|(|\Delta'| + ||\Delta'|||)^{O(1)} O(p)$, thus in $|V|^{2r+r'p} (|V|^{2r+r'p} + |V|^{r'})^{O(1)} O(p)$ and in $|V|^{O(1)}$, since r, r', p are all constants.

It is easy to observe that for a path from $(q'_0, \bar{0})$ to $(q'_f, \bar{0})$ of length $|V|^{O(1)}$ in $\mathcal{B}$, the maximum absolute value of the components of vectors along the path is at most polynomial in $|V|$, thus can be encoded in logarithmic space.

Therefore, to solve the original query evaluation problem, we can guess on-the-fly a path in $\mathcal{B}$ of length at most $|V|^c$ for some constant c , from $(q'_0, \bar{0})$ to $(q'_f, \bar{0})$, and record the latest vertex of G , the latest state of $\mathcal{A}$, and the latest constraint for the parameters $\bar{p}$, as well as the latest values of the vectors, all in logarithmic space. We conclude that the copyless- $\mathbb{Z}_{LA}$ -RPQC evaluation is in NL.

□